

Sprint 2 Grid Defense Script

Task 1 - Static Tests (PhpStan)

User / Role

What we did:

- ? Installed PhpStan and the Symfony/Doctrine extensions.
- ? Created a dedicated configuration file: 'phpstan.user-role.neon'.
- ? Ran a first analysis on the unoptimized version and captured the errors.
- ? Fixed the issues related to 'Users', 'Role', and their related classes.

What we fixed:

- ? Doctrine collection typing in 'Users' and 'Role'
- ? Uninitialized collections
- ? Safer type handling in user and role logic

What to say:

- ? For static testing, I used PhpStan with a dedicated configuration focused on my module.
- ? I first ran the analysis on the initial version, captured the errors, then corrected them and reran until the result was clean.

Result:

- ? Before optimization: real static-analysis errors were detected.
- ? After optimization: PhpStan returned no errors on the scoped configuration.

Joboffer / Application

What we did:

- ? Created a second PhpStan configuration: 'phpstan.joboffer-application.neon'.
- ? Analyzed 'Joboffer', 'Application', their controllers, forms, and related services.
- ? Fixed the detected issues and reran PhpStan until the scope was clean.

What we fixed:

- ? Collection typing
- ? Mapping and type mismatches
- ? Safer authenticated-user checks in controllers
- ? Cleaner business-rule conditions

What to say:

- ? For the second module, I used a separate PhpStan configuration so the analysis stayed scoped to 'Joboffer' and 'Application'.
- ? I followed the same workflow: before analysis, corrections, then clean after-analysis.

Result:

- ? Before optimization: multiple static issues were detected.
- ? After optimization: PhpStan returned no errors on this scope as well.

Task 2 - Unit Tests

User / Role

Chosen entity:

- ? 'Users'

What we did:

- ? Followed the workshop structure exactly.
- ? Generated a test with 'make:test TestCase UsersManagerTest'.
- ? Created a business service: 'UsersManager'.
- ? Implemented workshop-style unit tests in 'tests/Service/UsersManagerTest.php'.

Business rules tested:

- ? A local account must have a password.
- ? A local account password must contain at least 6 characters.
- ? An inactive role cannot be assigned.
- ? A Google-linked user may skip a password.

What to say:

- ? For the unit-test workshop, I chose the 'Users' entity, defined business rules, created a small service, generated the test class with 'make:test', and implemented the tests using 'TestCase'.

Result:

- ? More than 6 tests were implemented for this module.

Joboffer / Application

Chosen entity:

- ? 'Application'

What we did:

- ? Created a business service: 'ApplicationManager'.
- ? Implemented workshop-style tests in 'tests/Service/ApplicationManagerTest.php'.

Business rules tested:

- ? Application email must match the candidate email.
- ? Applications are only allowed for open job offers.
- ? Candidate experience must meet the job-offer requirement.
- ? Expected salary cannot be negative.
- ? Portfolio URL must be valid.
- ? Availability date cannot be before the application date.

What to say:

- ? For the second module, I chose the 'Application' entity and followed the same workshop approach: service plus 'TestCase' tests around explicit business rules.

Result:

- ? More than 6 tests were implemented for this module too.

Task 3 - DoctrineDoctor

What we did

- ? Installed 'ahmed-bhs/doctrine-doctor'.
- ? Registered the bundle in 'config/bundles.php'.
- ? Added the configuration in 'config/packages/doctrine_doctor.yaml'.
- ? Opened the relevant pages in dev mode through the Symfony profiler.

User / Role

Pages used:

- ? '/users'
- ? '/role'

What we fixed:

- ? Removed 'findAll()-style unbounded loading from the users listing flow.
- ? Removed ordering without limit.
- ? Reduced lazy-loading/repeated-entity-loading issues.
- ? Initialized missing collections in safe entity constructors.
- ? Added safe ORM cascade alignment in the owned scope.

What to say:

- ? I used DoctrineDoctor to detect runtime ORM and query issues on the admin pages, then I fixed the safe page-relevant problems and rechecked the profiler.

Result:

- ? The '/users' page improved from several performance issues to a much smaller remaining set.

Joboffer / Application

Pages used:

- ? '/joboffer'
- ? '/application'

What we fixed:

- ? Added result limits to sorted job-offer and application listings.
- ? Reduced unbounded query behavior.
- ? Initialized missing collections in safe entities.
- ? Added safe ORM cascade alignment in the owned scope.

What to say:

- ? On the job-offer and application pages, DoctrineDoctor highlighted ordering and loading issues. I applied low-risk optimizations that improved the listings without requiring schema-wide refactors.

Result:

- ? The main page-level performance warnings on '/joboffer' and '/application' were reduced.

Important justification

- ? I intentionally did not refactor all global architecture warnings, such as full snake_case renaming, full money-type redesign, or global blameable-trait adoption, because those changes affect unrelated modules and require larger database migrations.

Task 4 - Performance Report

What we measured

- ? Total execution time
- ? Peak memory usage
- ? DoctrineDoctor problem count before and after optimization

User / Role

Home page:

- ? '/joboffer'

Main feature:

- ? '/users'

What to say:

? I used Symfony profiler for response time and memory usage, and DoctrineDoctor for ORM/query issues.

? I compared the same pages before and after optimization.

Joboffer / Application

Home page:

? '/joboffer'

Main feature:

? '/application'

What to say:

? I used the application page as the main feature because it is the most representative workflow in the module.

? I compared before and after values for execution time, memory, and DoctrineDoctor warnings.

Task 5 - Scenario and Test Data

User / Role

Chosen story:

? As an admin, I want to manage users and roles so that the platform keeps correct permissions and account states.

Success case:

? Admin creates or edits a user with a valid active role and valid credentials.

Failure case:

? Admin tries to create a local account without a valid password.

Joboffer / Application

Chosen story:

? As a candidate, I want to apply to a job offer so that I can be considered by the recruiter.

Success case:

? Candidate applies to an open offer with valid data.

Failure case:

? Candidate applies to a closed offer, or tries to apply again to the same offer.

What to say:

? For each module, I prepared one clear user story with realistic data, one success case, and one failure case.

Task 6 - Subject Mastery and Justification

What to say:

? I used Symfony validators because business rules should be enforced on the server side.

? I used PhpStan to secure the code before runtime.

? I used workshop-style unit tests to validate business rules in isolation.

? I used DoctrineDoctor to detect ORM and query problems at runtime.

? For DoctrineDoctor, I prioritized fixes that were safe, measurable, and directly related to my modules.

Task 7 - Quantity of Work and Added Value

User / Role

Added value:

- ? AI user-role summary
- ? Google-linked account logic
- ? Face-recognition-related account handling
- ? QR contact export
- ? Role-based edit and back-navigation behavior
- ? Testing and Doctrine improvements

Joboffer / Application

Added value:

- ? Live search and sort
- ? Card-based listing UI
- ? Map-based location selection
- ? Resume upload
- ? AI resume autofill
- ? AI recruiter review
- ? Recruiter score and review note
- ? Duplicate-application prevention
- ? Email notifications
- ? Candidate delete-pending flow
- ? Testing and Doctrine improvements

What to say:

- ? The work goes beyond basic CRUD. It includes testing, performance optimization, validation, and advanced user-facing functionality.

Very Short Oral Version

Task 1

- ? I used PhpStan with a dedicated configuration for each module, captured the initial errors, fixed them, and reran the analysis until it was clean.

Task 2

- ? I followed the workshop style exactly: one chosen entity, one business service, one 'TestCase', and more than 6 tests per module.

Task 3

- ? I integrated DoctrineDoctor, captured the detected issues in the Symfony profiler, fixed the safe page-relevant problems, cleared the cache, and rechecked the result.

Task 4

- ? I used Symfony profiler for execution time and memory, and DoctrineDoctor for ORM and query issues, then compared the values before and after optimization.

Task 5

- ? I prepared one clear user story per module with logical test data, one valid case, and one failure case.

Task 6

? I can justify the use of Symfony validators, PhpStan, unit tests, and DoctrineDoctor according to correctness, maintainability, and performance.

Task 7

? The added value is that the work goes beyond CRUD through testing, optimization, advanced workflows, and AI-assisted features.